CSCI 6461 | Computer Systems Architecture | TOPIC 7

Machine Code & AArch64 Instruction Encoding

Instruction words, encoding fields, immediates, memory operations, and branch targets in AArch64

Dr. J. Burns

Topic 7 Outline

1. Machine Code and 32-bit Instruction Words
2. AArch64 Encoding Families
3. Data-Processing Encodings
4. Memory Instruction Encodings
5. Branch Instruction Encodings
6. Instruction Decoding Logic

Section 1

Machine Code and 32-bit Instruction Words

From Assembly to Machine Code

- A processor repeatedly **fetches**, **decodes**, and **executes** instructions.
- During fetch–decode–execute, the processor fetches a 32-bit instruction word from memory and interprets its bit fields.
- This topic starts with the earlier step: how an AArch64 assembly instruction is encoded as that fixed **32-bit** machine instruction.



In memory, `0x8B020020` is stored as bytes `20 00 02 8B` in increasing address order (A64 instruction byte order).

AArch64 Encoding Essentials

- Every A64 instruction occupies 32 bits (4 bytes), with operation and operand fields. Consecutive instruction addresses differ by 4 bytes.
- Different instruction families use different field layouts.
- Register fields are normally 5 bits, encoding register numbers 0–31.
- There are 31 general-purpose registers. Register number 31 denotes the zero register or stack pointer, depending on the instruction and operand.
- Identify the instruction form before interpreting its fields.

Section 2

AArch64 Encoding Families

How Instructions Are Encoded

An instruction's 32-bit word encodes the operation and its operands using a defined field layout.

- **Operation:** what to do, such as `add` or `ldr`. The bits identifying the operation are called the **opcode** (operation code).
- **Operands:** which registers or constant values to use.
- **Instruction format:** where the fields appear within the 32-bit instruction word. In AArch64, operation-identifying bits can span several fields.

Arm's ISA reference specifies each format's fixed bits: required patterns that help the decoder identify the format. The assembler fills the other fields from the chosen instruction and operands.

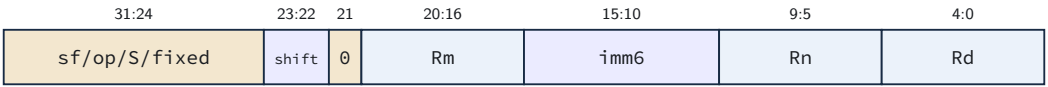
Section 3

Data-Processing Encodings

add/sub: Shifted-Register Form

Here we compare two versions of add: one uses the second source register directly; the other shifts its value left before adding.

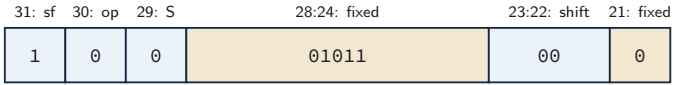
Sample instructions: `add x5, x6, x7` `add x5, x6, x7, lsl #3`



- `sf`: operand width; op: add/subtract; `S`: flag update. Fixed bits identify the format; the next slide gives their values.
- `Rd`: destination; `Rn`: first source; `Rm`: second source.
- Both examples use `shift=00` (lsl). The first implies `lsl #0`.
- `imm6` changes from `000000` to `000011` for `lsl #3`: the second example computes `x5 = x6 + (x7 « 3)`.

Deriving the Prefix: `add x5, x6, x7`

A **prefix** is a group of leading bits. Here, bits 31:21 combine fixed format bits with operation selectors:



Field	Choice	Reason
`sf`	`1`	64-bit `x` operands; `0` selects 32-bit `w` operands
op	`0`	Add; `1` selects subtract
S	`0`	Preserve flags; `1` updates them
Fixed bits	`01011` and `0`	Required bit pattern that identifies this instruction format to the decoder
`shift`	`00`	lsl; the absent shift defaults to lsl `#0`

Concatenate: `1 | 0 | 0 | 01011 | 00 | 0 = 10001011000`. The separate `imm6` field is `000000` for shift amount zero.

Common Arithmetic Register Patterns

All rows use 64-bit shifted-register forms: `sf=1`, `shift=00` (lsl), and bit 21 fixed at 0.

Instruction	op / S	Prefix 31:21
add	0 / 0	1|0|0|01011|00|0
adds	0 / 1	1|0|1|01011|00|0
sub	1 / 0	1|1|0|01011|00|0
subs	1 / 1	1|1|1|01011|00|0

- Prefix field order: `sf | op | S | 01011 | shift | 0`.
- op selects addition (0) or subtraction (1); `S=1` updates NZCV (negative, zero, carry, overflow flags).
- `imm6` selects the shift amount. For 32-bit forms, set `sf=0`; shift amounts are 0–31.

Encoding Example: `add x5, x6, x7`

Register Numbers

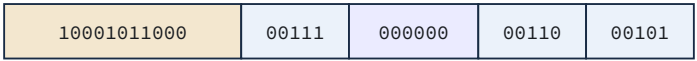
$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$Rm = x7 = 00111$$

Operation Fields

operation = add

operand size = 64-bit

shift amount = 0



The prefix comes from `sf/op/S/fixed/shift`; now insert the operands:

`10001011000 00111 000000 00110 00101` = `0x8B0700C5`.

add/sub: Immediate Form

Now we move on to the immediate form, where the second operand is a constant encoded in the instruction.

Sample instruction: `add x5, x6, #24`



- The immediate field is 12 bits wide.
- Bit 22, sh: 0 means no shift; 1 means shift the immediate left by 12 bits. Bit 23 is fixed at 0.
- This format is common for address arithmetic and small integer constants.

Common Arithmetic Immediate Patterns

add/sub (immediate) uses `sf | op | S | 100010 | sh | imm12 | Rn | Rd`. The required pattern `100010` at bits 28:23 identifies this format to the decoder; `sh` is bit 22.

64-bit form	op	S	Prefix 31:23
`add Xd, Xn, #imm`	0	0	1|0|0|100010
`adds Xd, Xn, #imm`	0	1	1|0|1|100010
`sub Xd, Xn, #imm`	1	0	1|1|0|100010
`subs Xd, Xn, #imm`	1	1	1|1|1|100010

Derive the add Immediate Prefix

`sf=1, op=0, S=0: 1 | 0 | 0 | 100010 = 100100010`.
For `#24`, choose `sh=0` and encode 24 directly in `imm12`. With `sh=1`, the immediate is shifted left by 12 bits.

Encoding Example: `add x5, x6, #24`

Operands

$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\text{imm12} = 24 = 000000011000$$

Instruction Meaning

```
x5 = x6 + 24
```

The immediate value is part of the instruction word.



`add x5, x6, #24` is encoded as `0x910060C5`.

Immediate rules depend on the instruction form. Constants outside this form may require another instruction form or a sequence of instructions.

Section 4

Memory Instruction Encodings

Scaled Offsets Depend on Access Size

In unsigned-offset loads/stores, the 12-bit `imm12` field stores the **byte offset divided by the access size**. The processor multiplies it back:

$$\text{address} = \text{base} + \text{imm12} \times \text{access size}$$

Instruction kind	Access size	Encoded immediate meaning
`ldrb/strb`	1 byte	byte offset
`ldrh/strh`	2 bytes	byte offset divided by 2
`ldr w/str w`	4 bytes	byte offset divided by 4
`ldr x/str x`	8 bytes	byte offset divided by 8

For 8-byte accesses:

- **Unsigned, scaled `imm12`:** 0 to +32,760 bytes, in steps of 8.

Scaling increases the addressable range without widening the immediate field.

Load/Store Instruction Encoding

Sample instruction: `ldr x5, [x6, #16]`



- `Rt` is the register loaded or stored.
- `Rn` is the base address register.
- `imm12` stores byte offset / access size in this unsigned-offset form.
- For a 64-bit `ldr`: address $=$ x6 $+ 8 \times$ imm12.

Memory Prefix: Width, Addressing Form and Operation

Bits 31:22 describe the transfer and help identify the addressing form. Read the fields separately before joining them to form the prefix.

Bits	Field	What it selects
31:30	`size`	Bytes transferred: `00` = 1, `01` = 2, `10` = 4, `11` = 8.
29:27	Fixed	Required pattern `111` for these load/store forms.
26	`VR`	`0`: general-purpose registers; `1`: SIMD and floating-point registers.
25	Fixed	`0`, part of the load/store group pattern.
24	Form	`1`: unsigned scaled offset. `0`: check additional bits for forms such as pre/post-index.
23:22	opc	Operation code: for the integer forms shown, `00` = store and `01` = load.

Example: `ldr x5, [x6, #16]` uses `11 | 111 | 0 | 0 | 1 | 01`.

The corresponding `str` changes only the last field to `00`.

Common Memory Instruction Patterns

Use the field order `size | 111 | VR | 0 | form | opc`. All rows use unsigned-offset addressing `[Xn|SP, #offset]`, with no base update.

Transfer	Bytes	size	opc	Prefix 31:22
`str Wt`	4	10	00	10|111|0|0|1|00
`ldr Wt`	4	10	01	10|111|0|0|1|01
`str Xt`	8	11	00	11|111|0|0|1|00
`ldr Xt`	8	11	01	11|111|0|0|1|01

Complete each word: `prefix | imm12 | Rn | Rt`. The encoded `imm12` equals byte offset divided by the number of bytes transferred.

Encoding Example: `ldr x5, [x6, #16]`

Operands

$$Rt = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\#16 = 2 \times 8 \text{ bytes}$$

Encoded Offset

$$\text{imm12} = 2 = 000000000010$$

The instruction stores the scaled value, not the byte offset directly.



`ldr x5, [x6, #16]` is encoded as `0xF94008C5`.

For `str x5, [x6, #16]`, keep the address fields and change opc from `01` to `00`: `0xF90008C5`. Now `Rt` is the source.

Offset, Pre-Index and Post-Index: What Changes?

Each example starts separately with `x6 = 0x1000`. All three load 8 bytes into `x5`. **Writeback** means updating the base register, `x6`.

Instruction	Address read	Final x6	Offset field
`ldr x5, [x6, #16]`	`0x1010`	`0x1000`	`imm12 = 2`
`ldr x5, [x6, #16]!`	`0x1010`	`0x1010`	`imm9 = 16`
`ldr x5, [x6], #16`	`0x1000`	`0x1010`	`imm9 = 16`

- **Offset:** add the offset for the access; leave the base unchanged. The unsigned `imm12` is scaled: $16/8 = 2$.
- **Pre-index:** update the base, then access the updated address.
- **Post-index:** access the old base address, then update the base.

Pre/post forms encode a signed **byte offset** in `imm9`: no division by access size. Their range is -256 to 255 bytes.

Addressing Modes: Encoding Reference

Reference layout: focus on offset scaling and base updates; exact mode-bit positions need not be memorised.

Unsigned offset: no base update



Pre/post index: base update



- **Bits 25 and 24:** Bit 25 stays `0`. Bit 24 is `1` for unsigned offset; the pre/post forms shown use `0` in both bit 24 and bit 21.
- **Mode bits 11:10:** `11` selects pre-index; `01` selects post-index.
- In unsigned-offset form, bits 11:10 are part of `imm12`; there is no separate mode field there.

Pre/Post-Index Changes: A32 and A64

A **stride** is the byte distance to the next item. A negative stride steps backward.

	A32 word `ldr`	**A64 integer `ldr`**
Immediate stride	12-bit magnitude; `U` sets direction. -4095 to $+4095$ bytes.	Signed 9-bit byte offset (`imm9`). -256 to $+255$ bytes.
Register stride	Pre: `ldr r0, [r1, r2]!` Post: `ldr r0, [r1], r2`	Separate `add x1, x1, x2` before (pre) or after (post) the load.

Post-index example: Read a 4-byte field from each 512-byte record.

A32: one instruction

```
ldr r0, [r1], #512
```

A64: 512 is out of range

```
ldr w0, [x1]
add x1, x1, #512
```

A64 still supports register offsets: `ldr w0, [x1, x2]` reads from `x1 + x2` and leaves `x1` unchanged.

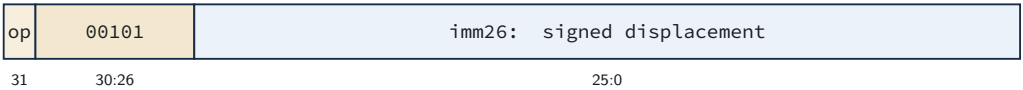
Section 5

Branch Instruction Encodings

Immediate Branch Instruction Layouts

Both layouts encode a signed displacement from the branch instruction's PC, in units of 4 bytes.

`b`/`bl`: `b target` or `bl function`



`b.cond`: e.g. `b.eq target`



- `b` and `bl` share five fixed bits. One `op` bit selects whether to save a return address, leaving 26 bits for the displacement.
- `01010100` identifies the immediate conditional branch format. That tells the decoder to read a 19-bit displacement and a separate four-bit condition. The shorter displacement limits the branch range.

Common Immediate Branch Patterns

PC is the branch instruction address. The signed displacement is an offset in 4-byte steps: target = PC + displacement $\times 4$.

b/bl use op | 00101: op=0 branches; op=1 also saves a return address. Required patterns 00101 and 01010100 identify the respective branch formats to the decoder.

Instruction	Fixed leading bits	Displacement field	cond (3:0)
b label	0|00101 (31:26)	imm26 (25:0)	—
bl function	1|00101 (31:26)	imm26 (25:0)	—
b.eq label	01010100 (31:24)	imm19 (23:5)	0000
b.ne label	01010100 (31:24)	imm19 (23:5)	0001
b.lt label	01010100 (31:24)	imm19 (23:5)	1011

- **Conditional branches:** required bit 4 = 0 also helps identify this form; cond selects EQ (Z=1), NE (Z=0), or signed LT (N differs from V).
- bl writes the return address, PC + 4, to x30.

Branch Example: Forward Target

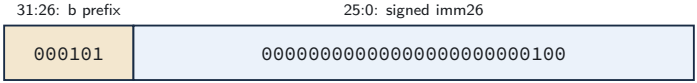
Reference PC = address of the branch itself. Encoded immediate = (target − PC) / 4.

Code Shape

```
0x100:  b target
0x104:  add x0, x0, #1
0x108:  sub x1, x1, #1
0x10c:  add x2, x2, #1
0x110:  target:  sub x3, x3, #1
```

Offset

$$\Delta = \texttt{0x110} - \texttt{0x100}$$
$$= 16 \text{ bytes}$$
$$\text{imm26} = 16/4 = 4$$



`b target`: positive displacement, `imm26=4`; word `0x14000004`.

Conditional Branch Example: `b.eq`

`b.cond` tests NZCV flags: signed `imm19` is scaled by 4; bits 3:0 select the condition, and bit 4 is fixed at 0.

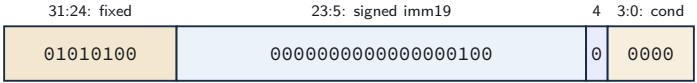
Target Offset

$$\text{current PC} = \texttt{0x100}$$
$$\text{target} = \texttt{0x110}$$
$$\text{imm19} = 4$$

Condition Field

$$\text{eq} = 0000$$

The branch is taken when the Z flag is set.



`b.eq target` is encoded as `0x54000080`.

Branch Example: Backward Loop

Assume `x0=0` and `x1` is a positive iteration count. The branch returns to an earlier instruction while `x1` is nonzero.

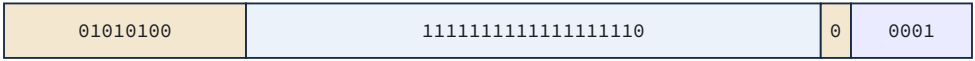
Loop Code (Instruction Addresses Shown)

```
0x200:  loop:  add x0, x0, #1
0x204:         subs x1, x1, #1
0x208:         b.ne loop
```

Signed Displacement

$$\text{PC} = \texttt{0x208}$$
$$\text{target} = \texttt{0x200}$$
$$\Delta = \texttt{0x200} - \texttt{0x208} = -8$$
$$\text{imm19} = -8/4 = -2$$

For -2: write $+2$ in 19 bits (17 zeros followed by `10`), invert all bits, then add 1. This gives `1111111111111111110` (`0x7FFFE`); `cond=0001` selects NE.



Check: `0x208 + (-2 * 4) = 0x200`. Instruction word: `0x54FFFFC1`.

Common Encoding Mistakes

- Treating a register number as decimal text instead of a binary field.
- Forgetting that AArch64 register fields are usually 5 bits.
- Encoding a byte offset directly when the immediate field is scaled.
- Adding a pipeline offset to PC instead of using the branch instruction address.
- Reading little-endian memory bytes as if they were the printed instruction word.

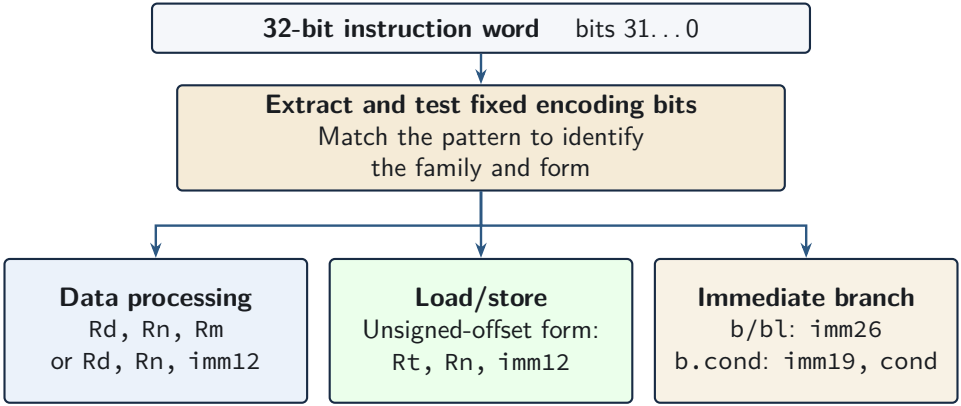
Practical Habit

Write the bit ranges first, then fill in the fields. Do not try to guess the final hexadecimal word too early.

Section 6

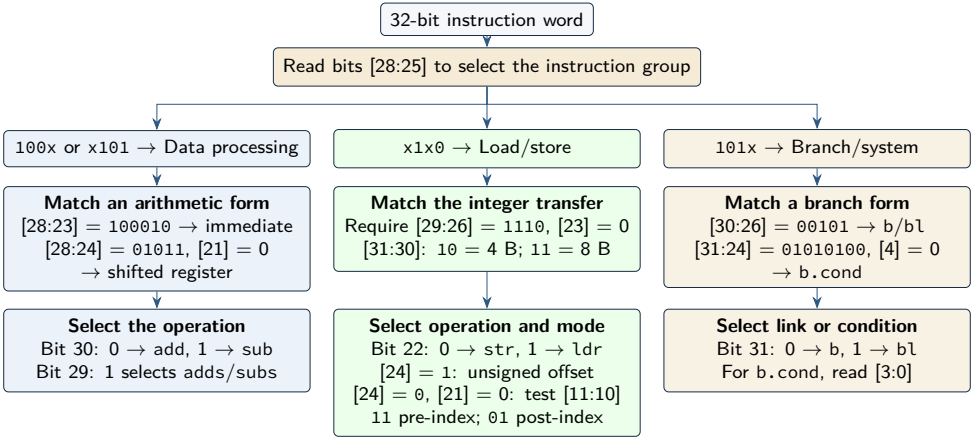
Instruction Decoding Logic

Instruction Decoding



Interpret operand fields only after identifying the instruction form.

Instruction Decoding: A Decision Tree



x means either bit value. Unmatched patterns go to other decode rules. Only forms taught here are shown.

Decoding Steps: A Worked Example

Start with the raw word `0xF90008C5`. Each result determines which bits to inspect next.

- **Route to a group.** Bits [28:25] are `1100`, which match `x1x0`: use the load/store path.
- **Match the transfer and address form.** Bits [29:26] are `1110` and bit 23 is 0. Bit 24 is `1`, so use the unsigned-offset layout.
- **Resolve the operation and width.** Bit 22 is 0 (`str`); bits [31:30] are `11` (8 bytes).
- **Interpret the operands.** Bits [9:5] give base `x6`; bits [4:0] give source `x5`. Bits [21:10] give `imm12=2`, so the byte offset is $2 \times 8 = 16$.

Decoded instruction: `str x5, [x6, #16]`.

Topic 7: Summary

- **Instruction words and forms:** A64 instructions occupy 32 bits (4 bytes), stored little-endian. Select the form, look up its fixed pattern, then interpret its fields.
- **Data processing:** op selects add/subtract and `S` controls NZCV updates. Register shifts use `shift`/`imm6`; arithmetic immediates use `imm12`/`sh`.
- **Memory:** unsigned offsets are scaled by access size. Pre/post-index forms use signed byte offsets and update the base; the mode determines which address is accessed.
- **PC-relative branches:** target = branch PC + signed immediate $\times 4$. `b`/`bl` use `imm26`; `b.cond` uses `imm19` and a 4-bit condition. `bl` saves PC + 4 in `x30`.

Self-Test Questions: Instruction Words

Use the reference tables; fixed binary patterns need not be memorised. Show how you choose the variable fields.

1. How many bytes does an A64 instruction occupy? For `0x8B020020`, list the four bytes in increasing memory-address order.
2. In the add/sub shifted-register form, what do `sf`, `op`, and `S` select? Which values encode a 64-bit subtraction that updates NZCV?
3. For `add x5, x6, x7`, give `Rd`, `Rn`, and `Rm` as 5-bit values. What happens to `shift` and `imm6` when `lsl #3` is added?
4. For `add x5, x6, #24`, give `imm12` and `sh`. What would setting `sh=1` do to the immediate value?
5. Split `10001011000` into its fixed and selector fields. Which selector bits change for `subs x5, x6, x7`?

Self-Test Questions: Memory and Branches

1. In `ldr x5, [x6, #16]`, why is `imm12=2`? Which field identifies the base register?
2. For an 8-byte access, compare the offset range and step size of unsigned scaled `imm12` and signed unscaled `imm9`. Which can address below the base?
3. Each time starting with `x6=0x1000`, compare the address read and final `x6` for offset, pre-index and post-index `ldr` with `#16`. Explain when the base register changes.
4. Compare `ldr x5, [x6, #16]` with `str x5, [x6, #16]`. What is the role of `x5` in each, and does either update `x6`?
5. A `b` at `0x100` targets `0x110`; a `b.ne` at `0x208` targets `0x200`. Calculate each signed immediate and give its field width. Which PC is the reference, and where is NE encoded?